

STDM: A Shared Tangent-Plane FP32 Divider–Multiplier with Multi-Level Accuracy–PPA Tradeoffs

Pingchuan Dong

Abstract—Division and multiplication are often implemented as separate approximate operators. This paper presents four STDM designs at accuracy levels L0–L3, each implementing approximate FP32 division and multiplication with a shared mantissa datapath. We derive the relation between their local tangent planes and express both operations as two residual products and a constant. Absorbing the divider’s reciprocal scale into these coefficients removes a subsequent multiplication. An exact recoding then replaces signed residual products and subtraction with unsigned products and addition. At each accuracy level, DIV and MUL use the same two short multipliers and one adder. In a TSMC 65-nm implementation, sharing at levels L1–L3 reduces area by 17.2–19.0%, power by 31.6–42.4%, and area–delay product by 11.8–14.4% relative to matched separate cores. DIV-only comparisons with PACE, FPD2D, and PLSAD show the remaining tradeoff between numerical accuracy and hardware cost. At L3, the complete shared unit uses 92.5% less area and 74.5% less delay than an exact DesignWare FP32 DIV+MUL unit.

Index Terms—approximate computing, floating-point arithmetic, divider, multiplier, tangent-plane approximation, arithmetic sharing

I. INTRODUCTION

Approximate arithmetic can reduce computation cost when an application tolerates numerical error. Division is a natural target because its mantissa computation is expensive. With most prior works, approximate dividers and multipliers are optimized separately; as a result, processing element that issues one operation at a time carries two mantissa cores.

This duplication is particularly costly in area-constrained accelerators whose kernels use multiplication for updates or accumulations, and need division for normalization or scaling. For example, K-means computes distances and coordinate sums with multiplications, then divides each sum by its cluster population to update the centroids [1]. Normalized adaptive filters similarly combine multiply-accumulate operations with a division by the input energy [2]. STDM solves this problem by taking advantage of the similarity in the approximation formulae for multiplication and division, reducing the overall area, power, and delay. The operation is selected independently for each input, but one STDM unit performs either division or multiplication, not both simultaneously.

STDM is built on piecewise tangent plane approximation, as inspired by OAM, which approximates multiplication around the midpoint of each mantissa cell [7]. We show that the divider tangent plane at the same midpoint uses the same residual terms, with one sign change and a reciprocal-square

scale. After absorbing this scale into the divider coefficients, both operations comprise two short products and a cell-selected constant.

Because the residuals are centered around zero, a direct implementation of this form would use signed multipliers and would subtract the y -residual product in DIV mode. STDM instead recodes the retained residuals as unsigned integers and adjusts the cell-selected constant to account for both the recoding offset and the DIV sign change. This exact transformation allows DIV and MUL to use the same two unsigned multipliers and one adder without changing the approximation.

STDM comprises four accuracy levels, L0–L3. The level is fixed when the circuit is generated; an input selects the operation. We evaluate both standalone dividers and matched separate and shared DIV+MUL cores.

The contributions are as follows:

- We derive a common piecewise tangent-plane formulation for approximate FP32 division and multiplication, revealing the arithmetic shared by the two operations.
- We develop four STDM designs that map this relation to a shared divider–multiplier datapath and provide distinct accuracy–cost tradeoffs.
- We analyze the approximation error and evaluate STDM against specialized dividers and matched separate DIV and MUL cores. At L1–L3, sharing reduces area by 17.2–19.0% and power by 31.6–42.4% without changing either operation’s output.

II. RELATED WORK

Approximate dividers differ in which part of division they simplify. Non-restoring and dynamic designs reduce the cost of quotient generation through approximate cells or pruning [8], [9]. TruncApp combines precision reduction with an approximate divider implementation [10]. These approaches retain much of the original arithmetic structure, so their cost depends strongly on the retained width.

Logarithmic designs replace division with subtraction in an approximate logarithm domain. FaNZed, LEAD, and CADE use compensation or modified approximations to improve the numerical result [11], [12], [13]. Among the works reviewed here, SIMDive is the closest approximate combined operator. It supports multiplication and division through shared Mitchell log-domain arithmetic [3], but processes 8–32-bit integers in SIMD FPGA lanes. In contrast, STDM is a scalar FP32 ASIC

design. Therefore, arbitrarily converting SIMDDive into an FP32 unit would introduce an adapter not evaluated in the original paper and would change both its numerical behavior and hardware cost. We therefore discuss SIMDDive as related work but do not use it as a PPA baseline.

Exact FP units have also combined multiplication and division to save hardware, but they solve a different problem. Chen *et al.* share signed-digit adders, a termination multiplier, and normalization stages in a single-precision fused unit [4]; unlike STDM, its target is an exact floating-point result rather than a selectable approximation level. The DIVA FPU reuses a pipelined mantissa multiplier across Taylor-series divider iterations [5], so its cost is tied to a sequential schedule. GRFPU similarly reuses multiplication hardware for iterative division and square root and supports both IEEE-754 single and double precision [6]. Its formats, operation set, and multicycle interface differ from STDM. These differences prevent a fair PPA comparison with STDM.

Reciprocal designs approximate $1/y$ and combine it with the dividend. SAADI-EC uses Taylor-series refinement, ILAFD combines reciprocal approximation with logarithmic multiplication, and QIAD uses quadratic interpolation [14], [15], [16]. PACE expresses its piecewise reciprocal computation through shifts, additions, and correction terms [17]. This avoids general multipliers in its divider mantissa core.

Surface-fitting designs approximate x/y directly. Earlier curve fitting, PLSAD, and FPD2D illustrate this approach [18], [19], [20]. PLSAD combines local fitting with simplified arithmetic; FPD2D optimizes quantized coefficients over a two-dimensional mantissa partition. They provide strong DIV-only baselines. STDM also evaluates a local affine function, but derives its coefficients from related product and quotient planes. That relation allows one datapath to serve both operations. The evaluation separates this architectural benefit from standalone divider efficiency.

III. DERIVATION AND SHARED ARCHITECTURE

For a normal finite FP32 input, write $F_x = (-1)^{s_x} x 2^{E_x}$ with $x \in [1, 2)$ [21]. MUL and DIV use the same sign XOR and differ in exponent addition or subtraction. We therefore focus on the mantissa functions xy and x/y .

A. Preliminaries: OAM

OAM partitions each axis of $[1, 2) \times [1, 2)$ into 2^n intervals at level n . The leading n fraction bits select indices i_x, i_y , whose cell midpoint is

$$\begin{aligned} k_x &= 1 + (i_x + \frac{1}{2})2^{-n}, \\ k_y &= 1 + (i_y + \frac{1}{2})2^{-n}. \end{aligned} \quad (1)$$

This midpoint minimizes the integral error of the local first-order product plane. With $r_x = x - k_x$ and $r_y = y - k_y$, OAM gives

$$\hat{m} = -k_x k_y + k_y x + k_x y = k_x k_y + k_y r_x + k_x r_y. \quad (2)$$

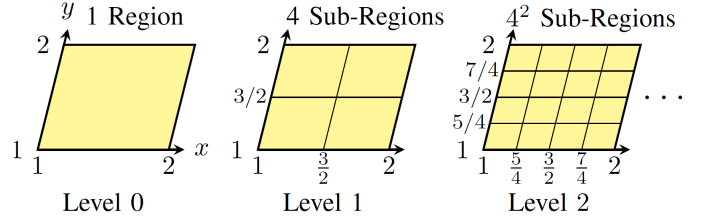


Fig. 1. Mantissa partition at successive levels. The leading fraction bits select a cell and its midpoint.

The residuals satisfy $|r_x|, |r_y| \leq 2^{-(n+1)}$. At L0, $k_x = k_y = 3/2$; at L3, each axis has eight intervals. Every L0–L3 midpoint has denominator at most 16 and is represented exactly.

B. Divider Tangent-Plane Equation Derivation

1) *Direct Derivation:* The first-order expansion of x/y at the same midpoint is

$$\begin{aligned} \hat{d} &= \frac{k_x}{k_y} + \frac{x - k_x}{k_y} - \frac{k_x(y - k_y)}{k_y^2} \\ &= \frac{k_x k_y + k_y x - k_x y}{k_y^2}. \end{aligned} \quad (3)$$

Substituting the centered coordinates and defining $c = k_y^{-2}$ gives

$$\hat{d} = c(k_x k_y + k_y r_x - k_x r_y). \quad (4)$$

Equations (2) and (4) contain the same constant and residual products. DIV changes only the sign of the y -residual term before applying c . Centering introduces no approximation; it only exposes the arithmetic shared by the two planes.

2) *OAM-Based Equivalent Derivation:* The reciprocal tangent at k_y is $g(y) = 2/k_y - y/k_y^2$. Replacing its product xy by the OAM plane gives

$$\begin{aligned} xg(y) &= \frac{2x}{k_y} - \frac{xy}{k_y^2} \\ &\approx \frac{2x}{k_y} - \frac{-k_x k_y + k_y x + k_x y}{k_y^2} \\ &= \frac{k_x k_y + k_y x - k_x y}{k_y^2}. \end{aligned} \quad (5)$$

This is identical to Eq. (3). The relation therefore follows algebraically from two expansions at the same midpoint, rather than from separately fitted DIV and MUL coefficient tables.

C. Architecture for Multi-Level Approximate Divider

The planes can be written recursively. If P_i is the plane at level i and $\Delta P_i = P_i - P_{i-1}$, then

$$P_n = P_0 + \sum_{i=1}^n \Delta P_i. \quad (6)$$

STDM instead decodes the selected cell and evaluates P_n once. The two forms are equivalent before precision reduction,

but direct evaluation avoids intermediate correction generators and additions.

The level fixes the partition, residual width, and coefficient precision when the circuit is generated. It is not a runtime input. The operation bit remains live and selects DIV or MUL coefficients at that level, so each reported unit supports either operation but produces one result at a time.

Centered residuals span one cell rather than the full mantissa range. Let $m_x = 2^{23}x$ be the unsigned 24-bit mantissa integer, and let j_x contain its lower $23 - n$ bits. Within a selected cell,

$$\begin{aligned} m_x &= 2^{23} + i_x 2^{23-n} + j_x, \\ R_x &= j_x - 2^{22-n}. \end{aligned} \quad (7)$$

Thus R_x has only $23 - n$ independent signed bits and requires no general midpoint subtractor; R_y is formed in the same way. Finer partitions reduce the product widths but require more coefficient entries, so both effects must be included in the hardware cost.

IV. SHARED CIRCUIT IMPLEMENTATION

A. Residual Precision and Coefficient Absorption

Let $R_x = 2^{23}r_x$ and $R_y = 2^{23}r_y$ be signed Q23 integers. Dropping D_r low bits produces

$$\rho_x = \lfloor R_x / 2^{D_r} \rfloor, \quad \rho_y = \lfloor R_y / 2^{D_r} \rfloor. \quad (8)$$

Each retained residual has $b = 23 - n - D_r$ signed bits. The arithmetic uses these short integers, while the interface remains FP32.

A straightforward divider would form the three-term plane and multiply its sum by a reciprocal coefficient. STDMM distributes that coefficient over the plane before generating RTL. Let $\bar{c}$ be a quantized reciprocal-square coefficient. With S fractional bits for each slope, define

$$\begin{aligned} A_M &= \text{round}(2^S k_y), & B_M &= \text{round}(2^S k_x), \\ A_D &= \text{round}(2^S \bar{c} k_y), & B_D &= \text{round}(2^S \bar{c} k_x). \end{aligned} \quad (9)$$

Here M and D identify the operation. Since the residual step is $\Delta_r = 2^{D_r-23}$, an integer product $A\rho$ represents $A\rho 2^{D_r-23-S}$. The constant must use the same scale, so its fractional precision is

$$F = S + 23 - D_r. \quad (10)$$

The two integer functions are

$$Q_M = T_M + A_M \rho_x + B_M \rho_y, \quad (11)$$

$$Q_D = T_D + A_D \rho_x - B_D \rho_y, \quad (12)$$

$$T_M = \text{round}(2^F (k_x k_y + b_M)),$$

$$T_D = \text{round}(2^F \bar{c} k_x k_y).$$

Their represented mantissas are $Q_M 2^{-F}$ and $Q_D 2^{-F}$. The output integer is shifted by $23 - F = D_r - S$ to restore Q23 alignment before normalization. This fixed shift requires wiring.

Residual truncation biases multiplication because both discarded terms have the same sign. Write $\tilde{r}_x = r_x - \epsilon_x$ and

$\tilde{r}_y = r_y - \epsilon_y$, with $0 \leq \epsilon_x, \epsilon_y < \Delta_r$. The plane error is $-k_y \epsilon_x - k_x \epsilon_y$. For inputs approximately uniform within a cell, $E[\epsilon_x] \simeq E[\epsilon_y] \simeq \Delta_r/2$. We therefore include

$$b_M = (k_x + k_y) \Delta_r / 2 \quad (13)$$

in T_M . It requires no separate correction adder.

Coefficient absorption removes the subsequent reciprocal multiplier, but rounding the absorbed coefficients can change the approximation. We treat that rounding separately from the exact recoding below. The constant and slopes are generated offline for each cell and mode. Synthesis implements their selection as combinational logic.

B. Exact Unsigned Residual Recoding

Signed products and DIV subtraction remain in Eqs. (11) and (12). Let $H = 2^{b-1}$ and introduce

$$u = \rho_x + H, \quad v = \rho_y + H. \quad (14)$$

Both fields range from zero to $2^b - 1$. Because each midpoint lies at the center of a binary interval, u and v are already present in the retained local fraction bits. For a fraction vector $f_x[22:0]$, $u = f_x[22-n:D_r]$; v is selected in the same way. No arithmetic is needed to construct them.

In fact, Eq. (7) gives $\rho_x = \lfloor j_x / 2^{D_r} \rfloor - H$. Adding H therefore recovers the retained local bits exactly. This also covers negative residuals: removing low bits implements a floor, not rounding toward zero. The distinction is essential to the constant adjustments in Eqs. (15) and (16). The DIV complement is applied only after selecting the retained field. It consequently needs b XOR gates rather than a complement over the full FP32 mantissa.

For multiplication, substitution gives

$$\begin{aligned} Q_M &= T'_M + A_M u + B_M v, \\ T'_M &= T_M - H(A_M + B_M). \end{aligned} \quad (15)$$

For division, use the b -bit complement $\bar{v} = (2H - 1) - v$. Since $\rho_y = H - 1 - \bar{v}$,

$$\begin{aligned} Q_D &= T'_D + A_D u + B_D \bar{v}, \\ T'_D &= T_D - H(A_D + B_D) + B_D. \end{aligned} \quad (16)$$

These are integer identities. The constant adjustments happen when the tables are generated, so the circuit only selects the adjusted constant. The selected L0-L3 coefficients yield nonnegative T' values. Both modes therefore use unsigned products and addition.

Figure 2 shows the common datapath. The cell indices and mode select A, B, T' . One unsigned multiplier receives u ; the other receives $v \oplus \{b\{m\}\}$, where $m = 1$ for DIV. Their products and T' feed the same adder. Alignment, normalization, exponent correction, and FP32 packing follow. The level-specific coefficient and residual widths are listed in Table I. DIV and MUL use identical precision settings within each shared unit.

For example, L0 has $H = 16$ and retains five bits per residual. MUL selects $(A, B, T') = (96, 96, 1632)$, while DIV selects $(44, 44, 760)$. Both sums are shifted left by 12 for Q23

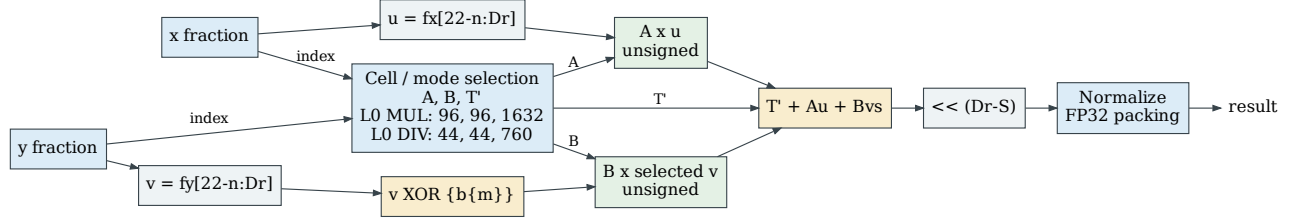


Fig. 2. Shared mantissa circuit. Mode $m = 1$ selects DIV and complements the retained y field; $m = 0$ selects MUL. All coefficient values are integer codes. The L0 example uses $D_r = 18$, $S = 6$, and $F = 11$.

TABLE I
SELECTED PRECISION OF THE SHARED MANTISSA CIRCUIT

Level	Residual drop D_r	Width b	S	F
L0	18	5	6	11
L1	16	6	4	11
L2	16	5	5	12
L3	16	4	4	11

alignment. The shared circuit thus changes coefficient values and complements five bits when the mode changes. It does not activate a separate divider core.

We checked the recoding against the signed integer forms over all retained residual combinations and cell entries. RTL and gate-level regressions then checked the packed outputs. The separate and shared implementations also produce identical outputs for both modes at each selected level. These checks establish equivalence of the implemented approximations; they do not imply exact FP32 multiplication or division.

C. Approximation Error and Coefficient Selection

Partition error, residual truncation, and coefficient quantization have different roles. For multiplication, the untruncated plane has the exact remainder $xy - \hat{m} = r_x r_y$, hence $|xy - \hat{m}| \leq 2^{-2n-2}$. For division, direct substitution gives

$$e = \frac{x}{y} - \hat{d} = \frac{r_y(k_x r_y - k_y r_x)}{k_y^2 y}. \quad (17)$$

Since $1 \leq k_x, k_y, y < 2$, the residual bound implies $|e| \leq 4^{-n}$. Thus each partition refinement reduces this conservative bound by a factor of four. It describes the tangent approximation before precision reduction.

With exact $c = k_y^{-2}$, residual truncation changes the divider error by

$$e^{(r)} - e = c(k_y \epsilon_x - k_x \epsilon_y). \quad (18)$$

The two contributions can cancel. A simple bound is $|e^{(r)} - e| < c(k_x + k_y)\Delta_r$. MUL instead adds both discarded contributions, which motivates Eq. (13). The compensation corrects their estimated mean, not each individual result.

To include coefficient error, define ideally scaled coefficients $A_D^* = 2^S c k_y$, $B_D^* = 2^S c k_x$, and $T_D^* = 2^F c k_x k_y$. Let $\delta_A =$

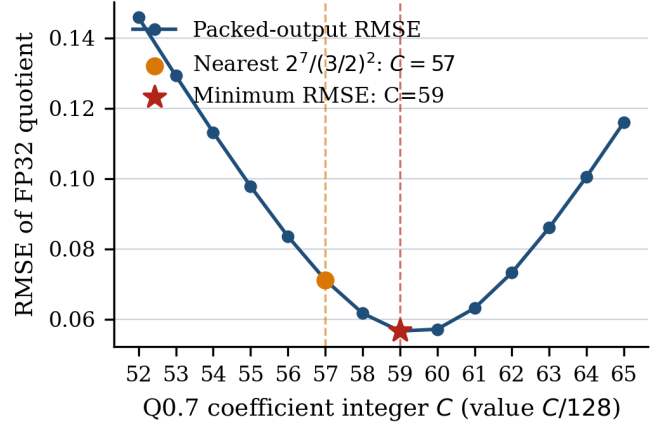


Fig. 3. L0 reciprocal calibration before coefficient absorption. Code C represents $C/128$. The minimum packed-output RMSE occurs at $C = 59$, rather than the nearest reciprocal code, $C = 57$. Final absorbed designs are evaluated separately in Table II.

$A_D - A_D^*$, $\delta_B = B_D - B_D^*$, and $\delta_T = T_D - T_D^*$. For the implemented integer coefficients,

$$e^{(q)} - e^{(r)} = -2^{-F}(\delta_T + \delta_A \rho_x - \delta_B \rho_y). \quad (19)$$

The minus sign follows from defining error as exact minus approximate. This expression includes reciprocal quantization and rounding after absorption. The H recoding contributes no additional error. Measured errors use the final FP32 outputs, including normalization and packing.

The nearest reciprocal code need not minimize the final error. For an intermediate plane w , let $e_0 = x/y - C_0 w$. Replacing C_0 by $C_0 + \delta$ gives

$$E[e^2] = E[e_0^2] - 2\delta E[e_0 w] + \delta^2 E[w^2]. \quad (20)$$

The cross term couples reciprocal error to plane error. Our earlier L0 calibration selects the Q0.7 code 59 rather than the nearest code 57. The stored integer 59 represents $59/128$; it is subsequently absorbed into A_D, B_D, T_D . This inherited choice is not a claim that 59 remains the unique optimum after every change in absorbed precision. The measured results below correspond to the selected final coefficients.

TABLE II
DIV-ONLY ACCURACY AND HARDWARE RESULTS. STDM, PACE, PLSAD, AND FPD2D SHARE THE 200,000-PAIR ACCURACY SET. ADP IS NORMALIZED TO EXACT DIV.

Design	MAE	MRED	RMSE	Area (μm^2)	Delay (ns)	Power (mW)	PDP (fJ)	Norm. ADP
Exact DesignWare DIV	–	–	–	19604.52	9.750	1.89509	18477.43	1.0000
STDM L0	0.042818	0.042856	0.056227	525.60	1.401	0.02103	29.47	0.0039
STDM L1	0.011796	0.011263	0.016233	809.64	1.685	0.03132	52.78	0.0071
STDM L2	0.003778	0.003568	0.005105	1030.32	1.940	0.03094	60.03	0.0105
STDM L3	0.002663	0.002577	0.003430	1149.12	2.006	0.03004	60.26	0.0121
PACE L1	0.028163	0.025951	0.045176	990.00	2.214	0.04467	98.92	0.0115
PACE L2	0.015143	0.014859	0.022305	1551.60	2.362	0.07829	184.88	0.0192
PACE L3	0.005897	0.005608	0.009281	2271.96	2.821	0.13119	370.08	0.0335
PACE L4	0.003856	0.003816	0.006060	3150.72	2.987	0.18531	553.51	0.0492
PLSAD ($m = 8$)	0.008975	0.008709	0.013227	659.88	1.738	0.02414	41.96	0.0060
FPD2D $8 \times 8, t = 17$	0.005344	0.005282	0.006828	790.56	1.873	0.01981	37.09	0.0077
FPD2D $8 \times 8, t = 16$	0.004054	0.004096	0.005134	869.76	1.962	0.02328	45.67	0.0089
FPD2D $8 \times 8, t = 15$	0.003653	0.003735	0.004636	942.12	2.038	0.02660	54.22	0.0100
FaNZeD	0.031811	0.031130	0.046191	2775.96	4.968	0.08880	441.16	0.0721
TruncApp	0.120105	0.119617	0.162391	4581.36	3.133	0.26136	818.77	0.0751
LEAD	0.021750	0.020993	0.028841	6666.12	5.674	0.60630	3440.38	0.1979
QIAD	0.005889	0.005880	0.006940	12097.80	8.074	0.95201	7686.66	0.5110

Figure 3 illustrates that calibration step. Once the reciprocal is absorbed, S controls a second quantization: it sets the grid on which the two slopes are represented. The constant uses the scale in Eq. (10), so changing S can also change its rounding. These changes need not move the errors in the same direction. For example, the selected L0 $S = 6$ point has slightly higher RMSE than the earlier $S = 7$ point, but lower MAE, MRED, and hardware cost. We retain this tradeoff rather than treating fewer coefficient bits as an automatic loss in every error metric.

Equation (19) also gives a bound over the retained integer domain:

$$|e^{(q)} - e^{(r)}| \leq 2^{-F}(|\delta_T| + H(|\delta_A| + |\delta_B|)). \quad (21)$$

The slope errors are weighted by the residual range. This explains why coefficient precision and residual precision must be selected together. A finer cell reduces that range, but may require more selection logic. The final choice uses measured error and synthesized cost; the bound identifies the error sources without predicting a unique hardware optimum.

V. EVALUATION

A. Method and Comparison Scope

We use Design Compiler U-2022.12-SP7 and PrimeTime U-2022.12-SP5 with the TSMC 65-nm typical CCS library. All circuits are combinational, with a 10-ns virtual clock, 20-ps I/O delays, `INVD0` input driver, and 0.004 output load. Power uses input probability 0.5 and toggle rate 0.1 per period. PDP and ADP denote power–delay and area–delay products; PDP is a vectorless estimate, not application energy.

PPA includes FP32 sign, exponent, normalization, and packing. The evaluated domain contains normal finite operands without exponent overflow or underflow; exceptional values

and subnormals are excluded. PACE retains the supplied mantissa RTL and normal-FP32 wrapper. PLSAD and FPD2D are local reconstructions. PLSAD uses ten-bit inputs, eight planes, and an eight-bit exact upper adder. FPD2D uses an 8×8 partition and reproduced coefficient optimization because its printed table contains anomalous cells. FaNZeD, TruncApp, LEAD, and QIAD retain the existing local adaptations; LEAD is a verified combinational unroll.

STDM, PACE, PLSAD, and FPD2D use the same 200,000 positive normalized FP32 pairs. Errors are measured after FP32 packing. Other divider rows retain their earlier 10,000-pair results for context; all PPA uses the common flow.

The exact combined reference contains parallel `DW_fp_div` and `DW_fp_mult` operators with an output selector. It provides the cost of supporting both exact FP32 operations under the same interface, rather than an accuracy-matched competitor. No prior approximate combined unit is placed in the table: SIMDiv has an integer SIMD FPGA interface, while the other fused units reviewed in Section II compute exact results with different timing contracts.

B. DIV-Only Accuracy and Cost

Table II shows the four STDM divider specializations. Accuracy improves from L0 to L3 in all three reported metrics. L0 has the smallest area and delay, but is less accurate than PACE L1. STDM L1–L3 have lower MAE, MRED, and RMSE than PACE L2–L4, respectively, with lower area, delay, and power in each pair.

The more specialized surface-fitting dividers present a closer comparison. PLSAD lies between STDM L1 and L2 in error and occupies less area than L1. FPD2D $t = 16$ nearly matches L2 RMSE while using less area and power. STDM L2 is slightly faster and has lower MRED, but its ADP remains

higher. At L3, STDm improves all three error measures over the evaluated FPD2D points, at greater area and power than $t = 15$. These results do not establish a universal DIV-only advantage. They show the cost of the divider approximation that underlies the shared unit.

TABLE III
COMBINED DIV+MUL COMPARISON AND SHARING ABLATION. SEPARATE AND SHARED HAVE IDENTICAL OUTPUTS WITHIN EACH LEVEL. BOTH INCLUDE ONE COMMON NORMALIZATION AND FP32 OUTPUT PATH. PERCENTAGES IN SHARED ROWS ARE CHANGES RELATIVE TO SEPARATE AT THE SAME LEVEL.

Design	Structure	DIV MRED	MUL MRED	Area (μm^2)	Delay (ns)	Power (mW)	PDP (fJ)	ADP (μm^2 ns)
Exact DesignWare	DIV+MUL	–	–	24025.68	9.856	2.08421	20541.66	236793.50
STDM L0	Separate	0.042856	0.032453	911.88	1.649	0.03451	56.89	1503.44
	Shared	0.042856	0.032453	891.36 (-2.3%)	1.836 (+11.3%)	0.03415 (-1.0%)	62.71 (+10.2%)	1636.94 (+8.9%)
STDM L1	Separate	0.011263	0.007903	1429.20	2.005	0.05746	115.23	2866.09
	Shared	0.011263	0.007903	1183.32 (-17.2%)	2.136 (+6.5%)	0.03932 (-31.6%)	83.99 (-27.1%)	2527.95 (-11.8%)
STDM L2	Separate	0.003568	0.002687	1796.40	2.271	0.06416	145.71	4079.48
	Shared	0.003568	0.002687	1454.40 (-19.0%)	2.401 (+5.7%)	0.03797 (-40.8%)	91.17 (-37.4%)	3492.15 (-14.4%)
STDM L3	Separate	0.002577	0.001901	2196.72	2.374	0.06850	162.62	5215.28
	Shared	0.002577	0.001901	1794.96 (-18.3%)	2.509 (+5.7%)	0.03944 (-42.4%)	98.95 (-39.2%)	4503.29 (-13.7%)

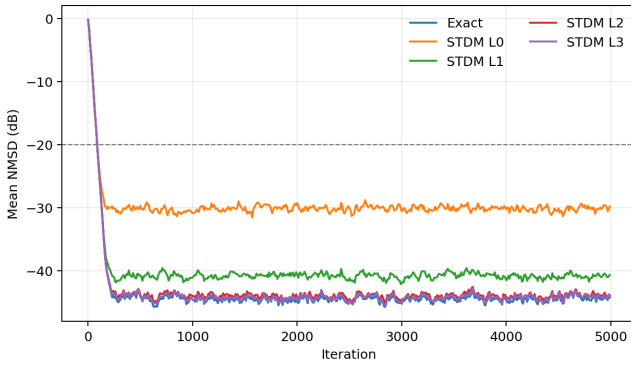


Fig. 4. Mean NLMS learning curves over 20 runs. NMSE is $10 \log_{10}(\|w_n - h\|_2^2 / \|h\|_2^2)$. The dashed line marks the -20 -dB convergence threshold.

C. Combined Unit and Sharing Benefit

Table III combines the full-unit comparison with the sharing ablation. In Separate, DIV and MUL have independent mantissa cores. A selector chooses their result before common normalization and FP32 packing. In Shared, cell decoding, both residual products, and accumulation are also shared. Both structures use coefficient absorption and H recoding with matching precision. This isolates the hardware effect of sharing without changing either operation’s outputs.

At L1–L3, sharing reduces area by 17.2–19.0%, power by 31.6–42.4%, and ADP by 11.8–14.4%. Delay increases by at most 6.5%. L0 behaves differently: area falls by 2.3% and power by 1.0%, but delay increases by 11.4%. Its ADP therefore rises by 8.9%. With only one cell, the separate L0 cores already use fixed coefficients. Mode-dependent coefficient selection adds overhead to the shared path. At higher levels, both structures need cell selection, and sharing saves a larger portion of the arithmetic. Separate is preferable at L0 when minimum ADP is the main objective.

At L3, STDM reduces area, delay, power, and ADP by 92.5%, 74.5%, 98.1%, and 98.1%, respectively, compared with exact DIV+MUL.

The measured benefit is strongest when one execution resource must support both operations without simultaneous issue. A design requiring independent DIV and MUL throughput would need separate resources or replicated units.

D. NLMS System Identification

We next test whether the arithmetic error affects an iterative application. An NLMS adaptive filter [2] estimates an unknown coefficient vector h from an input vector x_n and desired response d_n . With the current estimate w_n , one update is

$$\begin{aligned} \hat{d}_n &= w_n^T x_n, & e_n &= d_n - \hat{d}_n, \\ g_n &= \frac{\mu e_n}{\epsilon + \|x_n\|_2^2}, & w_{n+1} &= w_n + g_n x_n. \end{aligned} \quad (22)$$

The inner products and coefficient update use multiplication, while the normalization gain g_n requires division. A 16-tap update therefore performs 49 multiplications and one division.

We use 20 independent system-identification runs of 5,000 updates, with $\mu = 0.5$, $\epsilon = 10^{-3}$, and additive Gaussian noise of standard deviation 0.01. Each run starts from zero coefficients. The exact and STDM runs use identical unknown filters, input samples, and noise. STDM evaluates every normal finite multiplication and division with the corresponding L0–L3 RTL; additions remain exact FP32. In the application model, multiplication by zero and division with a zero numerator return zero. These cases lie outside the normal-finite wrapper used for PPA, account for 0.151% of the operations, and occur almost entirely while filling the delay line. No other out-of-scope operand or nonnormal result occurs.

All 20 runs converge at every level. Exact FP32 crosses the threshold after 139.8 updates on average; the STDM means range from 139.1 to 141.7. L2 and L3 raise steady-state output NMSE by only 0.53 and 0.27 dB, or 12.9% and 6.4% in mean-square error. L1 and L0 incur 3.59 and 13.54 dB, respectively, producing the level-dependent error floors in Fig. 4. L2–L3 therefore preserve the convergence behavior of the exact implementation.

REFERENCES

- [1] T.-W. Chen and S.-Y. Chien, "Bandwidth adaptive hardware architecture of K-means clustering for video analysis," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 18, no. 6, pp. 957–966, 2010, doi: 10.1109/TVLSI.2009.2017543.
- [2] C.-Y. Yao and Y.-Z. Huang, "The design of the NLMS adaptive filters using the fast-division approximation with CSD encoded divisors," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, 2024, doi: 10.1109/TC-SII.2023.3332692.
- [3] Z. Ebrahimi, S. Ullah, and A. Kumar, "SIMDive: Approximate SIMD soft multiplier-divider for FPGAs with tunable accuracy," in *Proc. GLSVLSI*, 2020.
- [4] C. Chen, H.-L. Yen, and M.-H. Sheu, "Design of a cost-effective multiplier/divider fused unit for floating-point arithmetic," *Int. J. Electr. Eng.*, vol. 11, no. 3, pp. 239–246, 2004.
- [5] T.-J. Kwon, J.-S. Moon, J. Sondeen, and J. Draper, "A 0.18- μm implementation of a floating-point unit for a processing-in-memory system," in *Proc. IEEE ISCAS*, 2004, vol. 2, pp. II-453–II-456, doi: 10.1109/ISCAS.2004.1329306.
- [6] Frontgrade Gaisler, "GRFPU: High-performance IEEE-754 floating-point unit," [Online]. Available: <https://www.gaisler.com/products/ieec-754-floating-point-unit>.
- [7] C. Chen, S. Yang, W. Qian, M. Imani, X. Yin, and C. Zhuo, "Optimally approximated and unbiased floating-point multiplier with runtime configurability," in *Proc. IEEE/ACM ICCAD*, 2020, pp. 1–9.
- [8] L. Chen, J. Han, W. Liu, and F. Lombardi, "Design of approximate unsigned integer non-restoring divider for inexact computing," in *Proc. GLSVLSI*, 2015, pp. 51–56, doi: 10.1145/2742060.2742063.
- [9] S. Hashemi, R. I. Bahar, and S. Reda, "A low-power dynamic divider for approximate applications," in *Proc. DAC*, 2016, pp. 1–6, doi: 10.1145/2897937.2897965.
- [10] S. Vahdat, M. Kamal, A. Afzali-Kusha, M. Pedram, and Z. Navabi, "TruncApp: A truncation-based approximate divider for energy efficient DSP applications," in *Proc. DATE*, 2017, doi: 10.23919/DATE.2017.7927254.
- [11] H. Saadat, H. Javaid, and S. Parameswaran, "Approximate integer and floating-point dividers with near-zero error bias," in *Proc. DAC*, 2019, doi: 10.1145/3316781.3317773.
- [12] O. G. Ratnaparkhi and M. Rao, "LEAD: Logarithmic exponent approximate divider for image quantization application," in *Proc. GLSVLSI*, 2022, doi: 10.1145/3526241.3530323.
- [13] M. Imani, R. Garcia, A. Huang, and T. Rosing, "CADE: Configurable approximate divider for energy efficiency," in *Proc. DATE*, 2019, pp. 586–589, doi: 10.23919/DATE.2019.8715112.
- [14] J. Melchert, S. Behroozi, J. Li, and Y. Kim, "SAADI-EC: A quality-configurable approximate divider for energy efficiency," *IEEE Trans. VLSI Syst.*, vol. 27, no. 11, pp. 2680–2692, 2019, doi: 10.1109/TVLSI.2019.2926083.
- [15] J. Oelund and S. Kim, "ILAFD: Accuracy-configurable floating-point divider using an approximate reciprocal and an iterative logarithmic multiplier," in *Proc. GLSVLSI*, 2023, pp. 639–644, doi: 10.1145/3583781.3590262.
- [16] H. Liu, M.-J. Wang, and M. Liu, "QIAD: A quadratic interpolation approximate divider," *IEICE Electronics Express*, vol. 20, no. 11, 2023, doi: 10.1587/elext.20.20230167.
- [17] C. Wen, H. Du, J. Wang, Z. Chen, L. Zhang, Q. Sun, and C. Zhuo, "PACE: A piece-wise approximate floating-point divider with runtime configurability and high energy efficiency," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 2, Art. 21, 2025, doi: 10.1145/3706634.
- [18] L. Wu, "A curve fitting approach for non-iterative divider design with accuracy and performance trade-off," in *Proc. NEWCAS*, 2015, doi: 10.1109/NEWCAS.2015.7182097.
- [19] C. Wu, W. Shi, Y. Yuan, Z. Zou, Z. Mo, and J. He, "Area-delay-energy-efficient approximate dividers based on piecewise linear fitting of surface," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 11, pp. 5017–5029, 2024, doi: 10.1109/TCSI.2024.3425538.
- [20] G. Di Meo, A. G. M. Strollo, D. De Caro, L. Tegazzini, and E. Napoli, "Low-power high precision floating-point divider with bidimensional linear approximation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 2, pp. 882–895, 2025, doi: 10.1109/TCSI.2024.3447830.
- [21] *IEEE Standard for Floating-Point Arithmetic*, IEEE Standard 754-2008, pp. 1–70, 2008.